PRESCRIPTION SEARCH SUPPORT

Architecture

Smals
ICT for society

RIZIV - INAMI

1. Contents

1. Contents 1
Document version 2
3. NIHDi PSS – High-Level Architecture 3
3.1. System Overview 3
3.2. Components and Responsibilities 4
3.3. Runtime Interaction (High Level) 6
3.4. Data and Persistence 6
3.5. Security and Access Control 6
3.6. Observability and Operations 6
3.7. Build and CI/CD 7
3.8. Module Interfaces Summary 7
3.9. Future Considerations 7
4. PSS Backend – Architecture Overview 8
4.1. System context and goals 8
4.2. High-level architecture 9
4.3. Module and package map 10
4.4. Request/response flow antimicrobial (happy path) 11
4.5. Request/response flow radiology (happy path) 12
4.6. Request/response flow biology (happy path) 13
4.7. DMN engine and decision evaluation 14
4.8. Persistence and data model 14
4.9. Error handling 16
4.10. Security and cross-cutting concerns 16
4.11. Configuration 16
4.12. Build, test, deploy, and run 16
4.13. API surface (high-level) 17
4.14. Architectural decisions and references 17

Document version

Version	Status	Date	Author	Description
1.0	Final	05/05/26	Smals	Initial version
1.1	Final	10/06/26	Nick, Fabrice	Add Clinical Biology

3. NIHDi PSS – High-Level Architecture

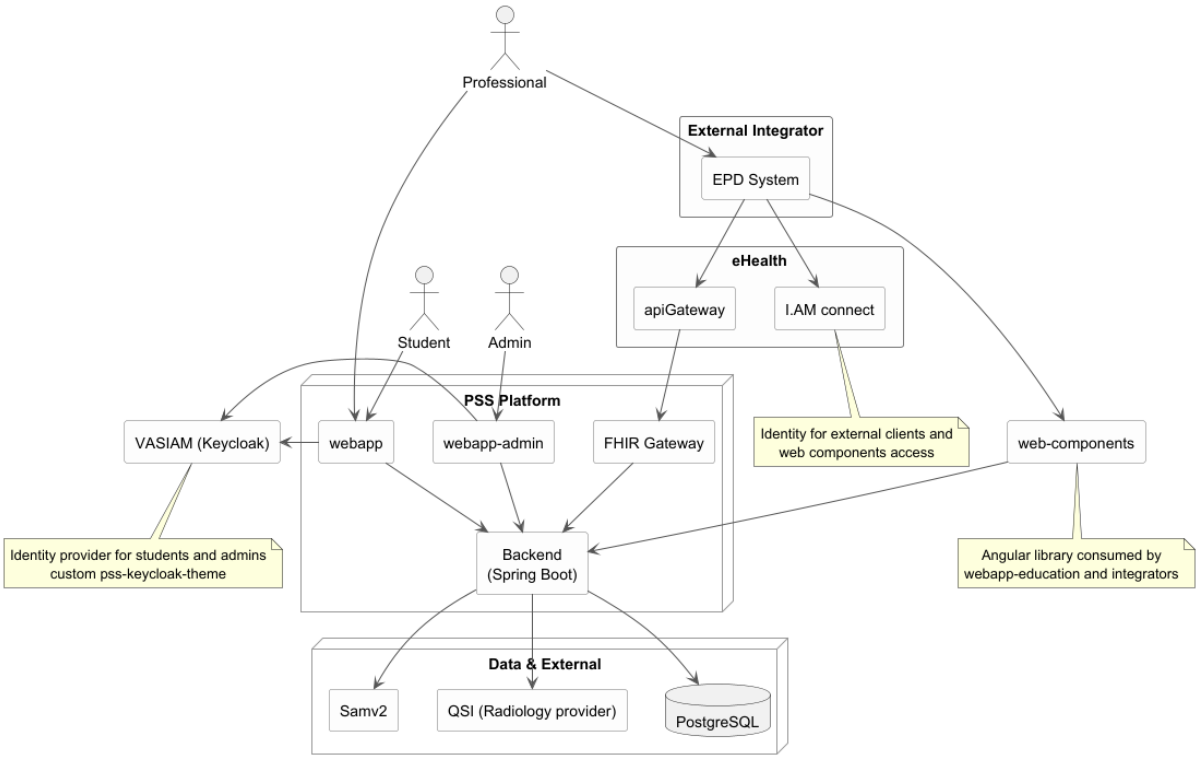
This document provides a concise, up-to-date, high-level view of the Prescription Support Service (PSS) platform. It summarizes the system's modules, their responsibilities, interactions, and key technology choices, and points to deeper documentation.

Scope: backend, fhir-gateway, pss-keycloak-theme, web-components, webapp-admin, webapp.

3.1. System Overview

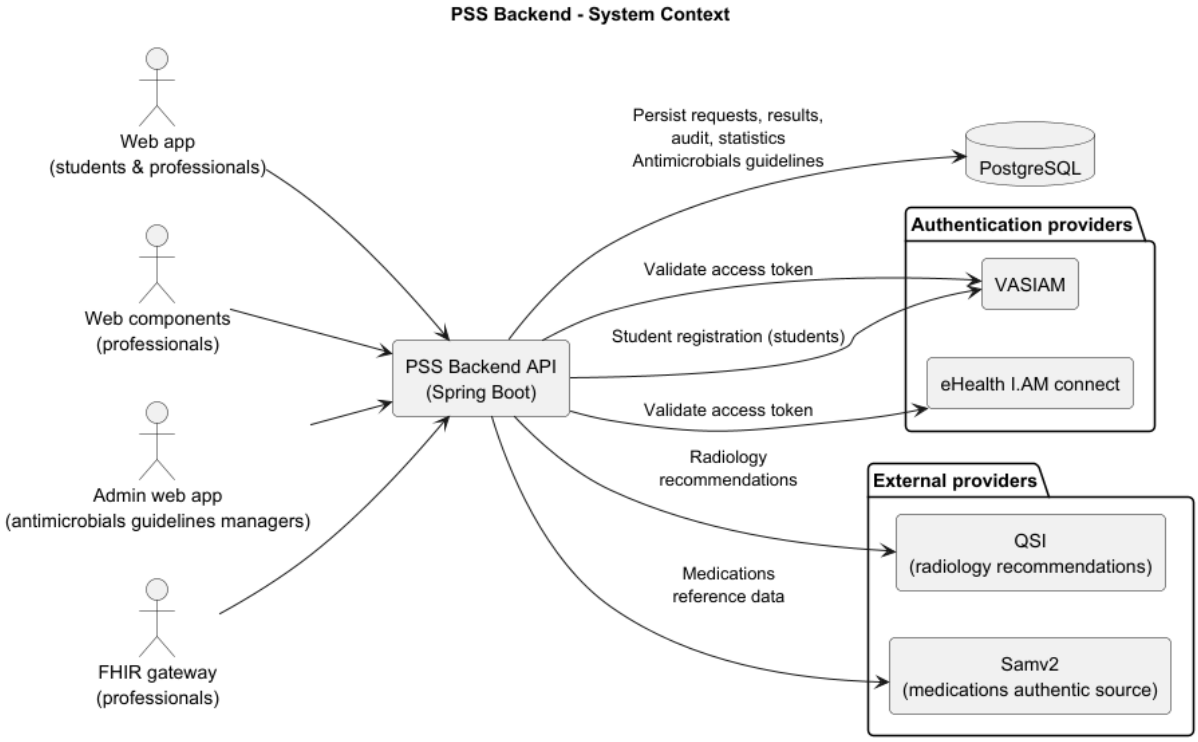
PSS delivers clinical decision support through REST and FHIR APIs, consumed by internal frontends and external systems. The platform is composed of:

- Backend: domain services, decision logic execution (DMN for antimicrobials and clinical biology, QSI integration for radiology), persistence, and REST APIs.
- FHIR Gateway: façade exposing a FHIR R4 API; delegates to backend.
- Frontends: Angular applications (Support and Admin) and a shared Web Components library.
- Identity and Access:
  - VASIAM (for webapp's) with a custom theme (pss-keycloak-theme).
  - eHealth I.AM connect (for fhir and web components)
- Data store: PostgreSQL managed via Flyway migrations.

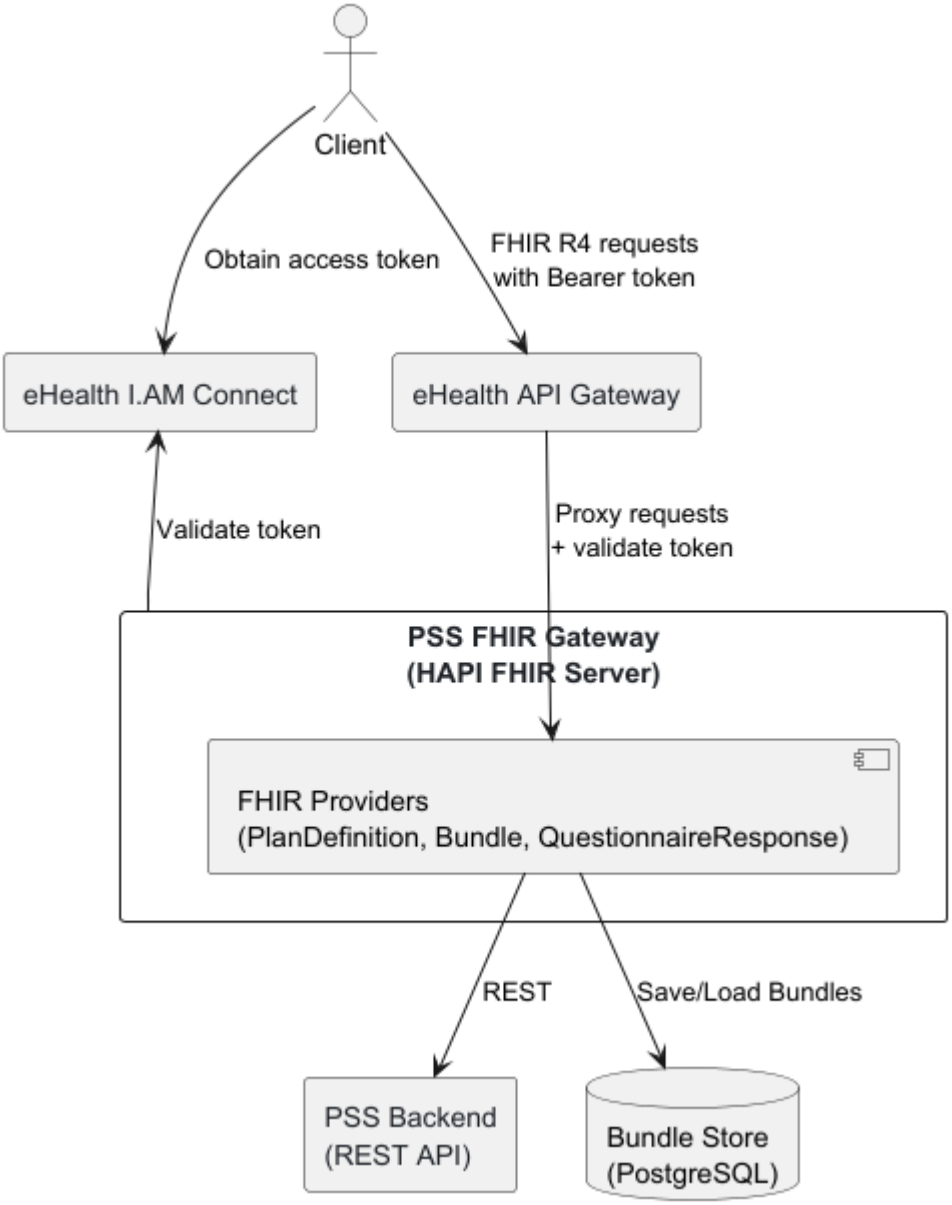


3.2. Components and Responsibilities

- backend
  - Type: Spring Boot service
  - Responsibilities:
    - Domain logic for antimicrobials, radiology, and clinical biology use cases
    - DMN evaluation (Camunda DMN 7.x)
    - Integrations (e.g., QSI for radiology)
    - Persistence (JPA/Hibernate, Flyway migrations)
    - REST APIs for admin and support flows
  - Key Tech: Java 21, Spring Boot 3.3, JPA/Hibernate, Flyway, Camunda DMN, MapStruct, Lombok
  - Interfaces: HTTP/REST (JSON), JDBC to PostgreSQL
  - Detailed doc: backend/docs/ARCHITECTURE.md



- fhir-gateway
  - Type: Spring Boot service
  - Responsibilities:
    - Expose FHIR R4 endpoints to external consumers
    - Delegate business operations to the backend
    - Handle OAuth2/OIDC with I.AM connect
  - Key Tech: Java 21, Spring Boot 3.3, HAPI FHIR 7 (R4), Web/WebFlux, OpenFeign
  - Interfaces: HTTP/REST (FHIR R4) to clients; HTTP to backend; OAuth2/OIDC to I.AM connect



- Webapp
  - Type: Angular SPA
  - Responsibilities:
    - Support and education workflows; consumes decision support APIs
    - State management (NgRx)
    - Auth via OAuth2/OIDC
  - Key Tech: Angular 19, NgRx, ngx-translate, Jest
  - Interfaces: HTTPS REST to backend
- webapp-admin
  - Type: Angular SPA
  - Responsibilities:
    - Administrative features and domain content management for antimicrobials
    - Auth via OAuth2/OIDC
  - Key Tech: Angular 20, Angular Material/Bootstrap, ngx-translate, Jest
  - Interfaces: HTTPS REST to backend
- web-components

    - Type: Angular library
    - Responsibilities:
        - Shared UI components and utilities consumed by webapp and integrators
    - Interfaces: N/A (library)
- pss-keycloak-theme
    - Type: Keycloak theme package
    - Responsibilities:
        - Custom identity provider look & feel for PSS
        - Interfaces: N/A (deployed to VASIAM Keycloak)

3.3. Runtime Interaction (High Level)

Typical flows:

- Webapp → Backend: Call to Backend support REST endpoints
- Webapp-admin → Backend: Call to backend admin REST endpoints.
- Web components → Backend: Call to Backend support REST endpoints
- External systems → eHealth Api Gateway → FHIR Gateway: External call FHIR R4 endpoints. The gateway calls backend support REST endpoints.
- Backend → External services: Backend may call QSI (radiology), and persists to PostgreSQL.

3.4. Data and Persistence

- PostgreSQL is the primary datastore.
- Backend uses JPA/Hibernate entities and repositories for runtime data (support requests, conclusions, radiology recommendations) and domain-managed content (indications, medications, translations, etc.).
- Flyway manages schema evolution with versioned migration scripts applied on startup.

3.5. Security and Access Control

- Identity/Access via VASIAM (Keycloak) (OAuth2/OIDC, JWT bearer tokens). (for the web app's)
    - VASIAM delegates to I.AM connect for professional users
- external web components & FHIR clients authenticate via I.AM connect in front.
- CORS is configured on backend to allow approved origins.
- Backend applies validation and centralized error handling for REST APIs.

3.6. Observability and Operations

- Health endpoints exposed for supervision.
- Application logs
- Metrics as provided by Spring Boot and custom business metrics where applicable.
- Containerization: Dockerfiles for services; deployment via Openshift/Jenkins pipelines.

3.7. Build and CI/CD

- Backend and FHIR Gateway: Maven builds; Docker images produced for deployment.
- Frontends and Web Components: Angular CLI builds.
- Jenkins is used for build pipelines and for scheduling batch/archival jobs where applicable.

3.8. Module Interfaces Summary

- Backend APIs (non-FHIR):
  - REST JSON endpoints for antimicrobials (support, admin), radiology (support, conclusions), clinical biology (support) and users.
  - See controllers and OpenAPI contracts in backend sources.
- FHIR Gateway APIs:
  - FHIR R4 resources (HAPI FHIR) as external interface; routes to backend.
- Authentication:
  - OAuth2/OIDC integration across frontends, gateway, and backend.

3.9. Future Considerations

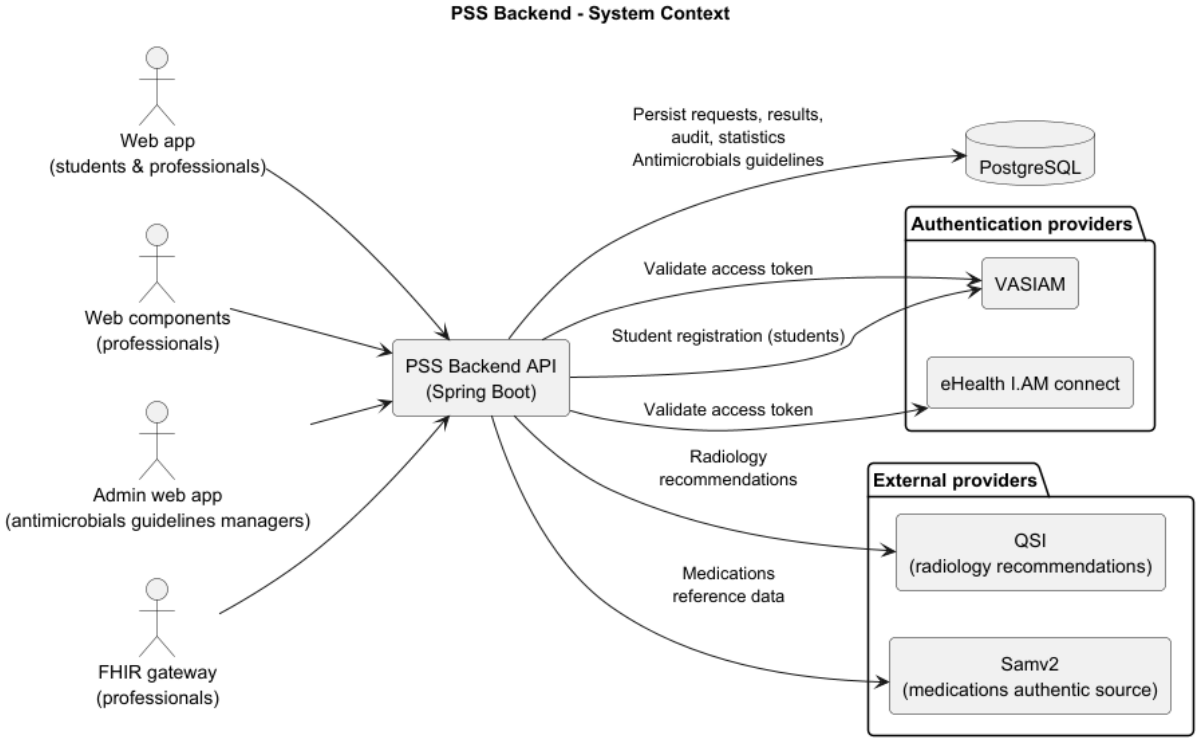
- DMN engine abstraction to allow swapping Camunda/KIE (see backend docs backend/docs/MIGRATING_DMN.md).
- Expanded observability (tracing sampling strategy) and metrics harmonization.
- Alignment of frontends to the same major Angular baseline when feasible.

4. PSS Backend – Architecture Overview

This document describes the architecture of the PSS backend service, its main components, data flows, technology choices, and operational concerns.

4.1. System context and goals

- Purpose: Provide clinical decision support and related services (e.g., antimicrobials, radiology, biology) via REST APIs used by frontend applications and/or integrating systems.
- Key capabilities:
  - Evaluate decision logic (DMN 1.3) to produce support recommendations and conclusions (antimicrobials).
  - Manage domain content (indications, medications, support options) and administrative operations (antimicrobials).
  - Integrates with external provider (QSI) to produce support recommendations and conclusions (radiology)
  - Evaluate decision logic (DMN 1.3) on laboratory tests to produce support recommendations for selected biology profiles (clinical biology).
  - Persist requests, conclusions, user statistics and audit information (antimicrobials & radiology).
- Non-functional goals: correctness and traceability of decisions, maintainability, and predictable performance.



4.2. High-level architecture

The backend is a Spring Boot application organized along a layered, hexagonal-inspired structure:

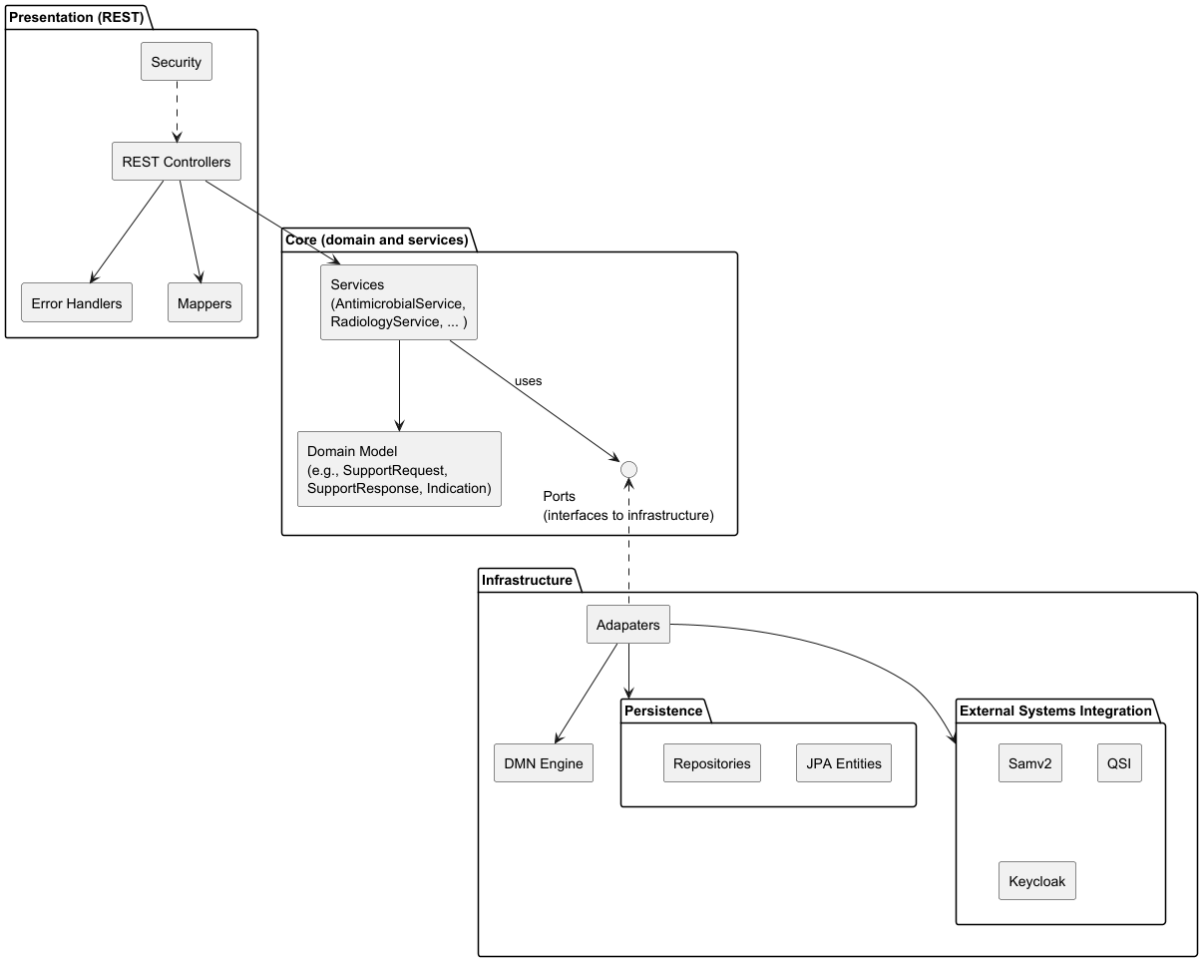
- Presentation layer (REST controllers)
- Core domain and services
- Infrastructure layer:
  - Persistence (JPA entities, repositories)
  - Antimicrobials & biology: DMN integration (decision models and evaluation engine)
  - External systems integration client
- Configuration/bootstrapping and cross-cutting concerns

Technology stack:

- Java 21 (as configured by Maven)
- Spring Boot 3.3 (web, validation, data)
- JPA/Hibernate for persistence
- Flyway for database migrations
- DMN evaluation engine: Camunda DMN 7.x (standalone) at present; see DMN migration plan
- Lombok for boilerplate reduction
- OpenAPI/Contracts

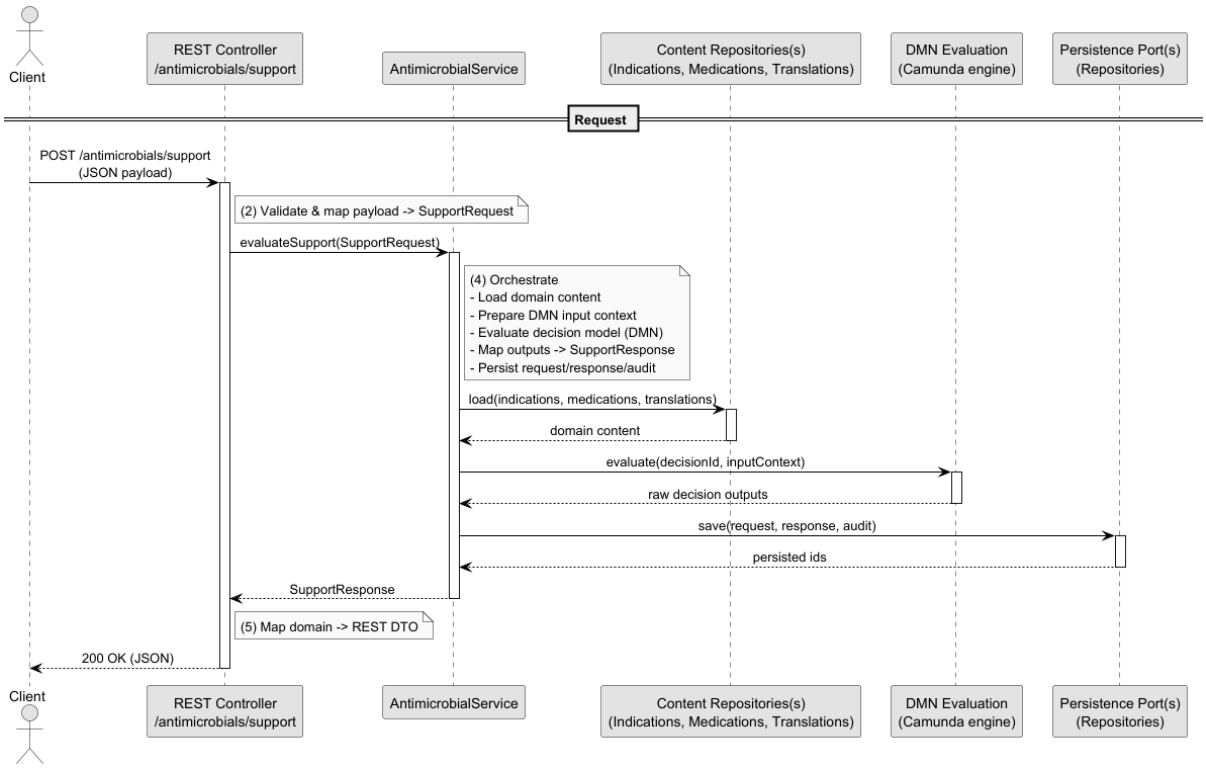
4.3. Module and package map

1. Application bootstrap
    - Spring Boot entry point
2. Presentation (REST)
    - Rest controllers
    - Error handlers
    - Mappers
    - Security
3. Core (domain and services)
    - Domain model: (e.g., SupportRequest, SupportResponse, Indication)
    - Services
    - Ports (interfaces) to infrastructure
4. Infrastructure
    - Adapters implement ports
    - Persistence entities and repositories
    - DMN engine integration: (see DMN engine notes below)
    - External systems integration clients (QSI, Samv2, Keycloak)
5. Database migrations
    - Flyway scripts



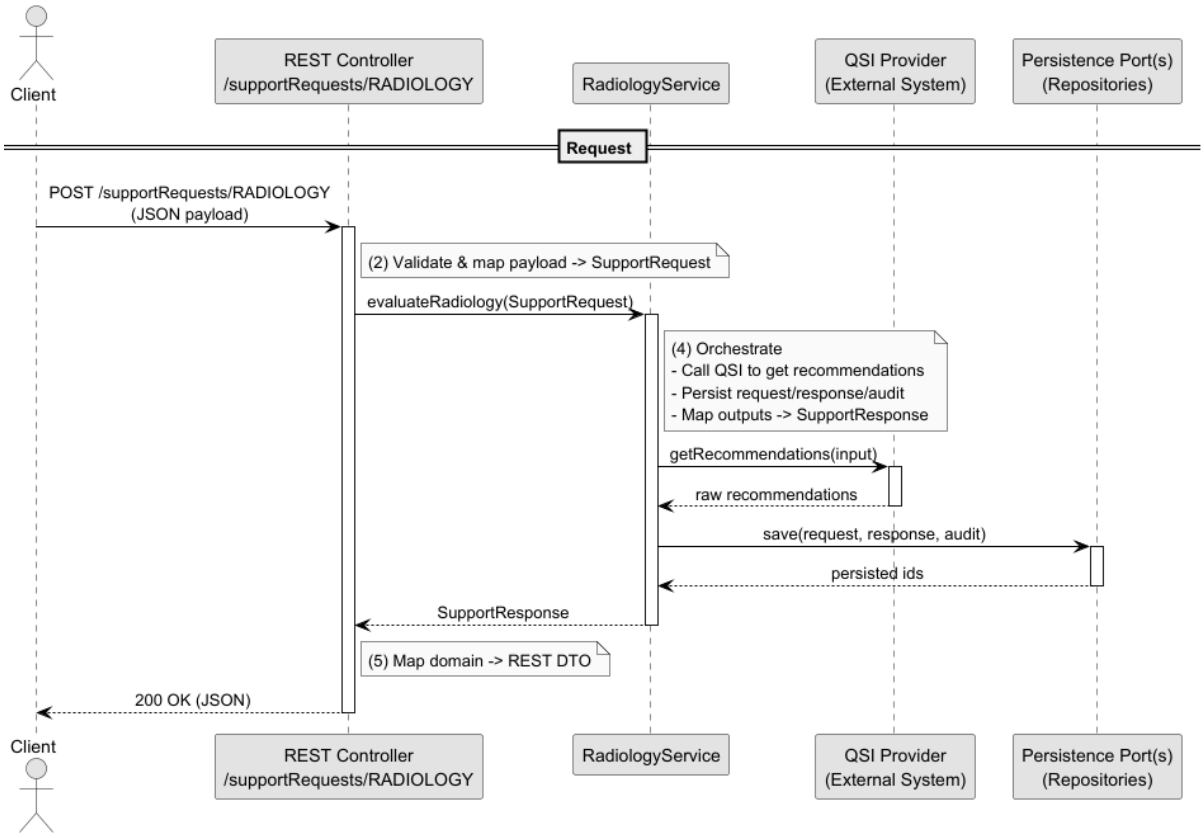
4.4. Request/response flow antimicrobial (happy path)

1. Client sends HTTP request to a REST endpoint (e.g., POST /antimicrobials/support).
2. Controller validates and maps incoming payload to domain types (e.g., SupportRequest).
3. Controller calls a core service (AntimicrobialService) through a clear domain API.
4. Core service orchestrates:
    - Loads referenced domain content (indications, medications, translations) via infrastructure ports.
    - Prepares DMN input context from the domain request.
    - Evaluates the decision model (DMN) via the DMN evaluation facility.
    - Maps raw decision outputs to domain response (SupportResponse) and business events.
    - Persists request/response/audit as required through persistence ports.
5. Controller maps SupportResponse to REST DTO and returns JSON.



4.5. Request/response flow radiology (happy path)

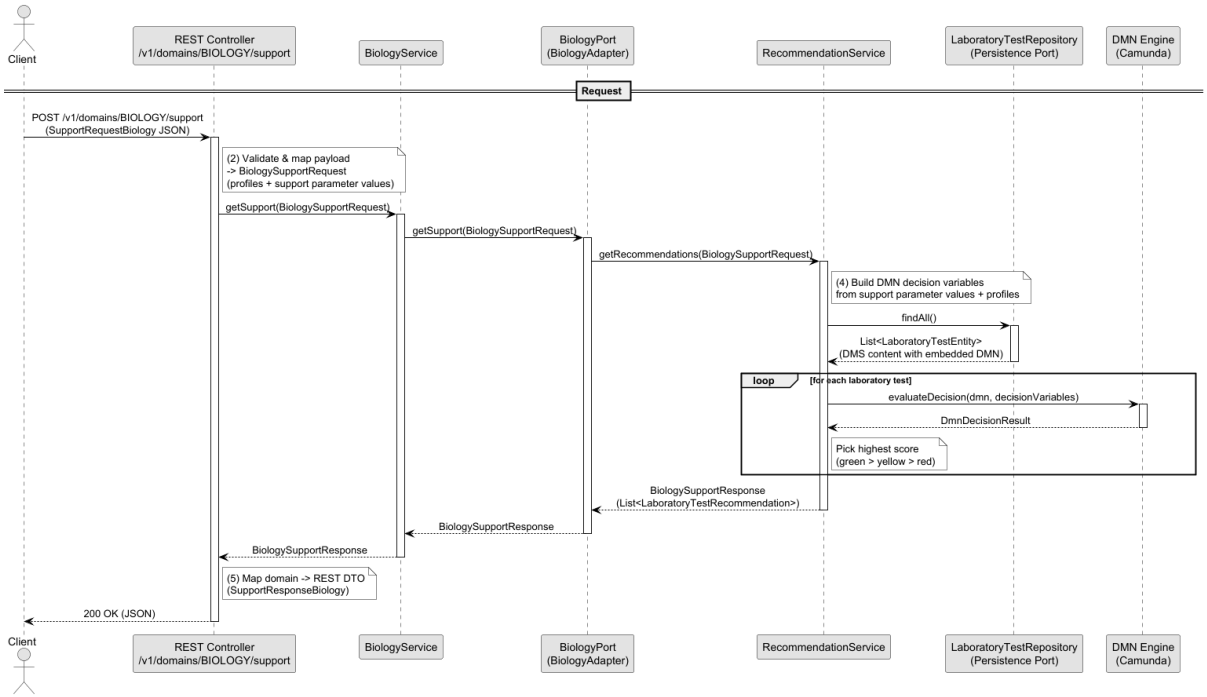
1. Client sends HTTP request to a REST endpoint (e.g., POST /supportRequests/RADIOLOGY).
2. Controller validates and maps incoming payload to domain types (e.g., SupportRequest).
3. Controller calls a core service (RadiologyService) through a clear domain API.
4. Core service orchestrates:
    - Calls QSI external provider to get support recommendations.
    - Persists request/response/audit as required through persistence ports.
    - Maps raw decision outputs to domain response (SupportResponse) and business events.
5. Controller maps SupportResponse to REST DTO and returns JSON.



4.6. Request/response flow biology (happy path)

1. Client sends HTTP request to a REST endpoint (POST /v1/domains/BIOLOGY/support).
2. Controller validates and maps incoming payload to domain types (BiologySupportRequest, with selected profiles and support parameter values).
3. Controller calls a core service (BiologyService) through a clear domain API.
4. Core service delegates to the BiologyPort (BiologyAdapter), which orchestrates via RecommendationService:
    - Builds the DMN input context (decision variables) from the request support parameter values and profiles.
    - Loads the laboratory tests (DMS content) and their embedded DMN models via persistence ports.
    - Evaluates the decision model (DMN) for each laboratory test via the DMN evaluation facility and keeps the highest score (green > yellow > red).
    - Maps raw decision outputs to domain response (BiologySupportResponse, a list of LaboratoryTestRecommendation).
5. Controller maps BiologySupportResponse to REST DTO (SupportResponseBiology) and returns JSON.

The biology domain also exposes supporting read endpoints: GET /v1/domains/BIOLOGY/profiles (available profiles) and GET /v1/domains/BIOLOGY/supportParameters (support parameters and their possible values for the selected profiles).



4.7. DMN engine and decision evaluation

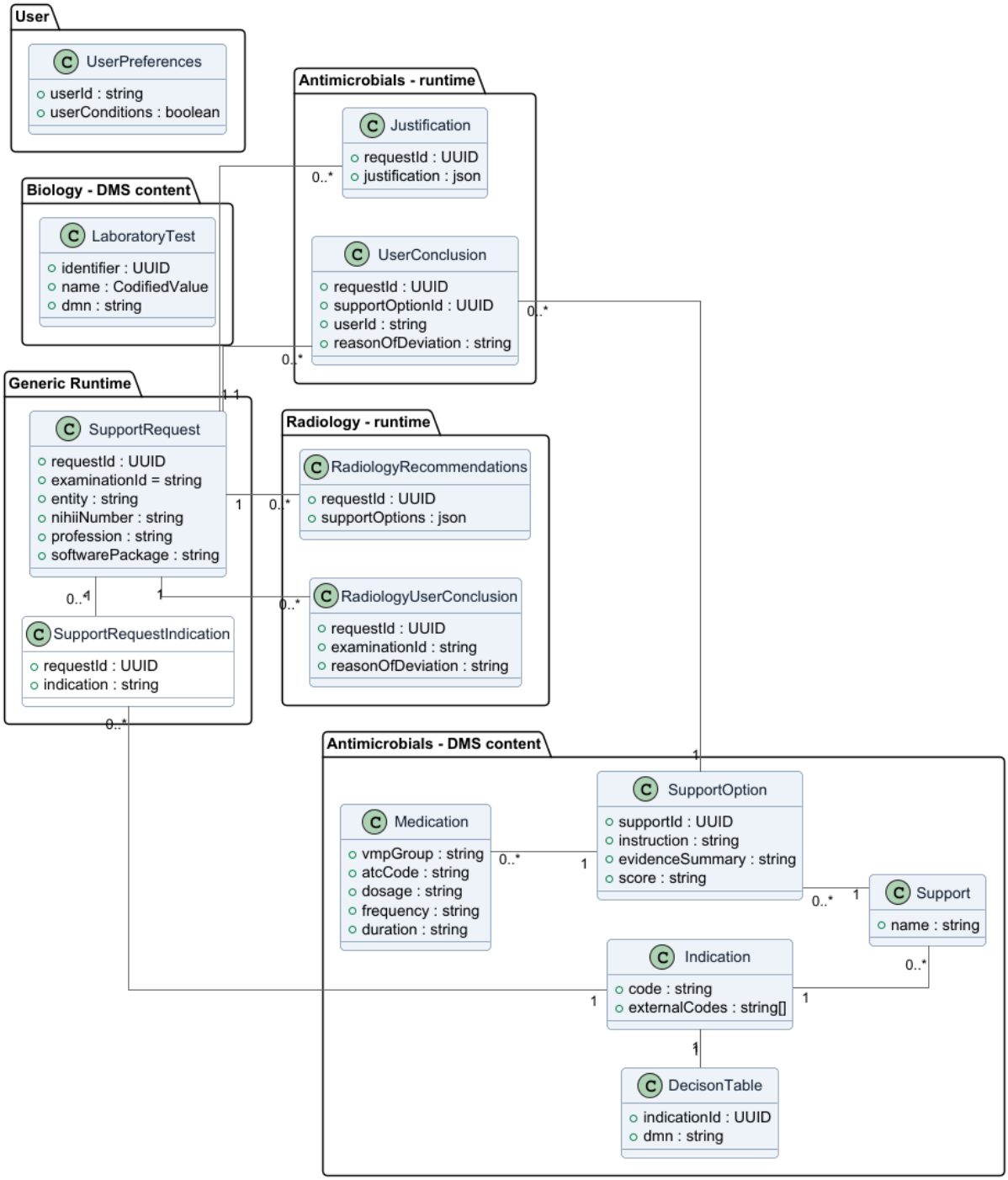
- Current engine: Camunda DMN (v7.19.0).
- Models: DMN 1.3 authored in Camunda Modeler.

DMN abstraction and migration plan:

- See docs/MIGRATING_DMN.md for a recommended refactor introducing DmnEvaluationService to decouple from Camunda and allow swapping to KIE DMN (or others) with a property (e.g., pss.dmn.engine=camunda|kie).
- Short term: keep behavior via a Camunda-backed adapter.
- Mid term: add KIE-backed adapter and validate parity across existing DMNs.

4.8. Persistence and data model

- JPA entities exist for:
  - Antimicrobials DMS content: IndicationEntity, MedicationEntity, SupportEntity, SupportOptionEntity, TranslationEntity, etc.
  - Generic Runtime/request data: SupportRequestEntity, SupportRequestIndicationEntity.
  - Antimicrobials runtime data: JustificationEntity, UserConclusionEntity
  - Radiology runtime data: RadiologyRecommendationsEntity, RadiologyUserConclusionEntity.
  - Biology DMS content: LaboratoryTestEntity (laboratory test with its embedded DMN model). Biology support requests/responses are evaluated on the fly and not persisted.
  - User preferences
- Versioning: common base such as VersionedEntity for DMS-managed versioned records.
- Database schema evolution managed by Flyway; each change is codified in a V<version>__<description>.sql script. Scripts are idempotent and applied on startup.


User
UserPreferences
userId : string
userConditions : boolean
Antimicrobials - runtime
Justification
requestId : UUID
justification : json
0..*
Biology - DMS content
LaboratoryTest
identifier : UUID
name : CodifiedValue
dmn : string
UserConclusion
requestId : UUID
supportOptionId : UUID
userId : string
reasonOfDeviation : string
0..*
0..*
Generic Runtime
1 1
SupportRequest
requestId : UUID
examinationId = string
entity : string
nihiiNumber : string
profession : string
softwarePackage : string
Radiology - runtime
RadiologyRecommendations
requestId : UUID
supportOptions : json
1
0..*
0..*
1
RadiologyUserConclusion
requestId : UUID
examinationId : string
reasonOfDeviation : string
SupportRequestIndication
requestId : UUID
indication : string
0..*
Antimicrobials - DMS content
1
SupportOption
supportId : UUID
instruction : string
evidenceSummary : string
score : string
Medication
vmpGroup : string
atcCode : string
dosage : string
frequency : string
duration : string
0..*
1
0..*
1
Support
name : string
0..*
Indication
code : string
externalCodes : string[]
1
1
1
DecisonTable
indicationId : UUID
dmn : string

4.9. Error handling

- Centralized REST error mapping:
  - SupportErrorHandler for support-related exceptions
  - UserErrorHandler for user-related exceptions.
- Validation errors are returned with appropriate HTTP status codes and messages aligned with controller method constraints.

4.10. Security and cross-cutting concerns

- CORS configuration through CorsConfiguration (allow-list and headers as per deployment needs).
- Input validation via Spring Validation annotations on request models/DTOs.

4.11. Configuration

- Application configuration via Spring properties/environment variables.
- DMN engine selection (future): camunda|kie as proposed in MIGRATING_DMN.md.
- Database connection, Flyway, logging, and CORS are configured via standard application-*.properties/yaml (see repo configuration files).

4.12. Build, test, deploy, and run

- Build: Maven project (pom.xml). Run mvn clean package to build.
- Unit and integration tests: under src/test/java; DMN assets under src/test/resources/dmn.
- Database: Local Postgres via docker-compose.yml for development; Flyway applies migrations on startup.
- Containerization: Dockerfile builds the service container.
- Configuration: Through Openshift config maps and secrets

4.13. API surface (high-level)

- Antimicrobials
  - Admin endpoints: content management operations
  - Support endpoints: submit SupportRequest and receive SupportResponse
- Radiology
  - Recommendation endpoints and user conclusions
- Clinical Biology
  - Profiles endpoint: list available biology profiles
  - Support parameters endpoint: parameters and possible values for selected profiles
  - Support endpoint: submit SupportRequestBiology and receive SupportResponseBiology (laboratory test recommendations)
- User
  - Create, delete, update users (for students)
  - Validate user consent
  - Set preferences

For exact contracts, check generated OpenAPI sources or controller method mappings.

4.14. Architectural decisions and references

- DMN engine strategy and migration: docs/MIGRATING_DMN.md.
- Database versioning strategy: Flyway with timestamped semantic versioned scripts.
- Coding style: layered architecture with ports-and-adapters flavor for domain services.